- This lab will cover recursion.
- It is assumed that you have reviewed chapters 4 of the textbook. You may want to refer to the text and your lecture notes during the lab as you solve the problems.
- When approaching the problems, think before you code. Doing so is good practice and can help you lay out possible solutions.
- Think of any possible test cases that can potentially cause your solution to fail!
- You must stay for the duration of the lab. If you finish early, you may help other students. If you don't finish by the end of the lab, we recommend you complete it on your own time. Ideally you should not spend more time than suggested for each problem.
- Your TAs are available to answer questions in lab, during office hours, and on Edstem.

Vitamins (25 minutes)

---

1. For each of the following code snippets:
    a. Given the following inputs, trace the execution of each code snippet. Write down all outputs in order and what the functions return.

    b. Analyze the running time of each. For each snippet:
        i. Draw the recursion tree that represents the execution process of the function, and the cost of each call
        ii. Conclude the total (asymptotic) run-time of the function.

a.
```
def func1(n):                #  Draw out func1(16)
    if (n <= 1):
        return 0
    else:
        return 10 + func1(n-2)
```

b.
```
def func2(n):                #  Draw out func2(16)
    if (n <= 1):
        return 1
    else:
        return 1 + func2(n//2)
```

c.
```
def func3(lst):              # Draw out func3([2, 4, 6, 8, 10, 12])
    if len(lst) == 1:
        return lst[0]
    else:
        return lst[-1] + func3(lst[:-1])
```

```
d. def func4(n):              # Draw out func4(64)
       if n <= 1:
           return 1
       else:
           return n + func4(n // 2)

e. def func5(n):              # Trace func5(5)
       if n <= 0:
           return 0

       total = 0
       for i in range(n):
           total += i

       return total + func5(n - 1)
```

Coding

In this section, it is strongly recommended that you solve the problem on paper before writing code.

1. Write a **recursive** function that returns the **sum** of all numbers from 0 to n. (5 minutes)

```
def sum_to(n):
    """
    : n type: int
    : return type: int
    """
```

2. Write a **recursive** function that returns the product of all the even numbers from 1 to n **(assume n is passed in as an even number)**. (5 minutes)

```
def product_evens(n):
    """
    : n type: int
    : return type: int
    """
ex)  if n = 8 → 2 * 4 * 6 * 8 = 384
```

3. **Sum of Digits**
   Write a recursive function that returns the sum of the digits of a positive integer n. You must solve this problem without casting the input into a string.

```
def digit_sum(n):
    """
    : n type: int
    : return type: int
    """
```

ex)
```
digit_sum(5724) # 18
digit_sum(8)    # 8
digit_sum(101)  # 2
```

Hint: Consider using `n % 10` and `n // 10` in your solution.

4. **Is the List Strictly Increasing?**
   A list is strictly increasing if every element is smaller than the element immediately after it. Write a **recursive** function that determines whether a non-empty list is strictly increasing.

```
def is_increasing(lst):
    """
    : lst type: list
    : return type: bool
    """
```

ex)
```
is_increasing([2, 5, 8, 13]) # True
is_increasing([2, 5, 5, 9])  # False
is_increasing([7])           # True
```

5. **Count Adjacent Matches**
   Write a recursive function that returns the number of times two equal values occur next to each other in a list.

```
def count_adjacent(lst, low, high):
    """
    : lst type: list
    : low type: int
    : high type: int
    : return type: int
    """
```

ex)
`count_adjacent([1, 1, 3, 5, 5, 5], 0, 5) # 3`
`count_adjacent([1, 2, 3, 4], 0, 3) → 0`

Explanation:
The matching adjacent pairs are (1, 1), (5, 5), and (5, 5) so the result is 3
There are no matching adjacent pairs in the second example, so the result is 0

6. **Valid Palindrome (Leetcode 125)**
   A palindrome is a phrase where all characters are the same backward and forward. (must be recursive)

   Given a string `str` and its range of indices to consider, return `True` if it is a palindrome, or `False` otherwise. Assume all characters are lowercase. Must run in O(n) time.

ex)

```
is_palindrome("racecar", 0, 6) returns True      # racecar
is_palindrome("racecar", 1, 5) returns True      # aceca

is_palindrome("race car", 0, 6) returns False
# (empty space counts as a character)

is_palindrome("racecar", 1, 3) returns False     # ace
```

```
def is_palindrome(str, low, high):
    """
    : str type: str
    : low, high type: int
    : return type: bool
```

```
    """
```

7. **Binary Search (Leetcode 704)**
   Given an array of integers `nums` which is sorted in ascending order, `low` and `high` indices, and an integer `target`, write a function to search for `target` in `nums`. If `target` exists between `low` and `high`, then return its index. Otherwise, return `None`.

   You must write an algorithm with O(log n) runtime complexity. (must be recursive)

```
def binary_search(lst, low, high, val):
    """
    : lst type: list[int]
    : val type: int
    : low type, high type: int
    : return type: int (found), None
    """
```

8. **Count Peaks**
   A peak is any element in a list strictly greater than both of their immediate neighbors. Write a **recursive** function that counts how peaks are in a list `lst`. Do not consider the first or last element to be peaks.

```
def count_peaks(lst, low, high):
    """
    : lst type: list[int]
    : low type: int
    : high type: int
    : return type: int (found), None
    """
```

   ex)
   `count_peaks([2, 7, 3, 8, 5, 9, 1], 0, 6) # 3`

   Explanation: 7, 8, and 9 are peaks because they are strictly greater than their adjacent elements.

9. Vowel and Consonant Count

Given a string of letters representing a word(s), write a **recursive** function that returns a **tuple** of 2 integers: the number of vowels, and the number of consonants in the word.

Remember that tuples are not mutable so you'll have to create a new one to return each time when you're updating the counts. Since we are always creating a tuple of 2 integers each time, the cost is constant. Implementation run-time must be linear. (25 minutes)

```
def vc_count(word, low, high):
    """
    : word type: str
    : low, high type: int
    : return type: tuple (int, int)
    """
```

ex)

```
word = "NYUTandonEngineering"
vc_count(word, 0, len(word)-1) → (8, 12)
#  8 vowels, 12 consonants
```